



Получите новый уровень в карьере

Повышайте свою квалификацию



Хабр



Чему научиться в этом году?



MishaBucha

21 час назад

Легаси и минус 99% времени: пошаговый разбор оптимизации

👤 Средний 🕒 12 мин 👁 4.9K

Java*, Kotlin*

Кейс

Всем привет! Меня зовут Михаил, я главный эксперт в ОТП Банке.

Думаю, многие из вас сталкивались с легаси, которое нужно дорабатывать и оптимизировать. Сегодня хочу поделиться реальным кейсом, как мы ускорили отправку данных в смежную систему.

Разберем всё по шагам, с замерами производительности. Поехали!

Задача: У нас есть пользователи, которые регистрируются в нашей системе. Нужно также отправить их на регистрацию в систему партнера. Сервис партнера находится вне нашего контура, и единственный способ интеграции REST-вызовы.

Казалось бы, можно взять и отправить пачку записей. Но не тут-то было. Партнер принимает только по одной записи за раз. У них много других партнеров, и они боятся не выдержать нагрузки.

Значит, нам нужно отправлять тысячи пользователей по одному, но сделать это максимально быстро и без потери надежности. Вдобавок асинхронность использовать нельзя, да и работает всего один инстанс. Совсем печально.

Наш легаси код и замеры его производительности

Конечно, тут будет код не реальной системы, а учебной, но суть будет точно такой же:

```
@Transactional
public void sendUserToRegistration() {
    Stopwatch stopWatch = new Stopwatch();
    stopWatch.start();

    List<User> savedUsers = new ArrayList<>();
    Pageable pageable = PageRequest.of(0, 2000, Sort.by("id").ascending());
    Page<User> page;
    do {
        page = userRepository.findAllByStatus(UserStatus.NEW, pageable);
        page.getContent().forEach(user -> registerAndAddToSaveList(user, savedUsers));
        pageable = page.nextPageable();
    } while (page.hasNext());

    userRepository.saveAll(savedUsers);
    stopWatch.stop();
}
```

```

        System.out.println(stopWatch.prettyPrint());
    }

    private void registerAndAddToSaveList(User user, List<User> savedUsers) {
        RegistrationDto registrationDto = userMapper.toRegistrationDto(user);
        RegistrationResponseDto response = otherSystemClient.registrationUser(registrationDto);
        UserStatus userStatus = UserStatus.valueOf(response.getStatus());
        user.setStatus(userStatus);
        savedUsers.add(user);
    }

```

[Объяснить](#) с  SourceCraft

Чтобы внешняя система не упала от нагрузки, они попросили присылать им пользователей в определенное окно, когда меньше всего запросов летит к ним, это 5 утра.

Обычный шедулер:

```

@Scheduled(cron = "0 0 5 * * *") // один день в 5 часов
void sendRegistration() {
    log.info("Начало отправки пользователей на регистрацию");
    processService.sendUserToRegistration();
    log.info("Конец отправки пользователей на регистрацию");
}

```

[Объяснить](#) с 

Данные и скорость ответа внешней системы

Количество данных очень важно, так как есть системы, которые принимают и работают с 2-3 пользователями в час и нагрузки нет никакой. Мы же представим, что у нас 75к записей в базе. В нашем тестовом проекте не будет настоящего похода во внешнюю систему, мы ее будем эмулировать следующим методом:

```

public RegistrationResponseDto registrationUser(RegistrationDto dto) {
    LockSupport.parkNanos(TimeUnit.MILLISECONDS.toNanos(1));
    return new RegistrationResponseDto(UserStatus.SUCCESS.name(), null);
}

```

[Объяснить](#) с 

Сам этот метод нам сейчас не очень важен, мы представим, что внешняя система отвечает нам 1 мс(что в реальности может быть ооооочень далеким от правды).

Результаты выполнения легаси

Мое железо: Apple M4 процессоров + 16 оперативной памяти

Первый запуск:

```

StopWatch '': 104.962466417 seconds
-----
Seconds      %      Task name
-----
104.9624664  100%

```

[Объяснить](#) с 

Начальная точка $\approx$ 105 секунд

Шаг 1. Конечно индексы

Сразу все наверное сказали, добавь индекс и не парь мозг, делаем:

```
CREATE INDEX idx_user_status_id ON public.users (status, id);
```

[Объяснить с](#) 

Видим точно такой же результат, данных не так много, да и пачка не очень большая

```
StopWatch '': 105.086878875 seconds
```

```
-----  
Seconds      %      Task name  
-----  
105.0868789  100%
```

[Объяснить с](#) 

Шаг 2. Поменять логику сохранения в бд

Как мы видим в нашем коде логика сохрани лежит тут:

```
userRepository.saveAll(savedUsers);
```

[Объяснить с](#) 

И тут есть несколько корневых проблем, предлагаю включить show-sql и глянуть что там:

```
spring:  
  jpa:  
    show-sql: true
```

[Объяснить с](#) 

Видим запросы для findAllByStatus:

```
Hibernate: select u1_0.id,u1_0.email,u1_0.is_active,u1_0.name,u1_0.status from users u1_0  
Hibernate: select count(u1_0.id) from users u1_0 where u1_0.status=?
```

[Объяснить с](#) 

Из-за Page spring генерирует сразу 2 запроса, давайте заменим Page на Slice:

```
StopWatch '': 104.810421 seconds
```

```
-----  
Seconds      %      Task name
```

104.810421 100%

[Объяснить с](#) 

Тоже недалеко ушли(

Но на самом деле это не то, что я хотел показать, вся боль у нас тут(причем это после `findAllByStatus` и не дойдя до `saveAll`):

```
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
```

[Объяснить с](#) 

Если у кого-то есть вопросы почему так - объясню:

Когда мы делаем:

```
userRepository.findAllByStatus(UserStatus.NEW, pageable);
```

[Объяснить с](#) 

Мы загружаем их через Hibernate и даем ему управлять их состоянием

Затем мы меняем объект

```
user.setStatus(userStatus);
```

[Объяснить с](#) 

И снова делаем

```
userRepository.findAllByStatus(UserStatus.NEW, pageable);
```

[Объяснить с](#) 

Мы загружаем сущности через Hibernate, и они попадают в **persistence context** (managed state).
Hibernate делает **flush перед SELECT**, следовательно до каждого запроса получаем 2000 запросов на обновление, что печально.

Получается, что наш **savedUsers** хранит 75 записей в конце и все это в памяти, так еще и все равно мы сохраним все до метода **saveAll**:

```
List<User> savedUsers = new ArrayList<>();
...бла бла бла
userRepository.saveAll(savedUsers);
```

Меняем код на:

```
@Transactional
public void sendUserToRegistration() {
    Stopwatch stopwatch = new Stopwatch();
    stopwatch.start();

    Pageable pageable = PageRequest.of(0, 2000, Sort.by("id").ascending());
    Slice<User> page;
    do {
        page = userRepository.findAllByStatus(UserStatus.NEW, pageable);
        List<User> content = page.getContent();
        content.forEach(this::registerUser);
        userRepository.saveAll(content); // не держим в памяти и сохраняем сразу
    } while (page.hasNext());

    stopwatch.stop();
    System.out.println(stopwatch.prettyPrint());
}

private void registerUser(User user) {
    RegistrationDto registrationDto = userMapper.toRegistrationDto(user);
    RegistrationResponseDto response = otherSystemClient.registrationUser(registrationDto);
    UserStatus userStatus = UserStatus.valueOf(response.getStatus());
    user.setStatus(userStatus);
}
```

Объяснить с 

Мда, ничего не поменялось:

```
StopWatch '': 104.769344167 seconds
-----
Seconds      %      Task name
-----
104.7693442  100%
```

Объяснить с 

Но подождите, смотрим запросы и снова видим кучу запросов вместо 1:

```
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
Hibernate: update users set email=?,is_active=?,name=?,status=? where id=?
и тд
```

Объяснить с 

Есть несколько способов это решить, я же выберу просто написать @Query в репозитория:

```

@Modifying
@Query(value = """
    UPDATE users SET status = CAST(:status AS text)
    WHERE id IN (:ids)
    """, nativeQuery = true)
int batchUpdateStatus(@Param("ids") Collection<Long> ids, @Param("status") String sta

```

[Объяснить с](#) 

Наконец-то получаем прирост производительности $\approx 6\%$

```
StopWatch '': 98.409089208 seconds
```

```
-----
Seconds      %      Task name
-----
```

```
98.40908921  100%
```

[Объяснить с](#) 

Шаг 3. Играемся с пачками

Количество пачек тоже очень важно, его нужно выбирать с умом, слишком мало = много запросов в бд, слишком много = долгие запросы, давайте в нашем примере тоже поиграемся.

ВАЖНО, серебрянной пули в выборе пачки нет - все очень субъективно, зависит от железа, куда хотите дать нагрузку и тд, так что всегда пробуйте разные варианты)

Сейчас имея пачку в **2000** наш результат составляем:

```
StopWatch '': 98.409089208 seconds
```

```
-----
Seconds      %      Task name
-----
```

```
98.40908921  100%
```

[Объяснить с](#) 

Давайте начнем с уменьшения пачки, поставим например 500 и получаем результат $\approx 8.41\%$ хуже:

```
StopWatch '': 106.68422925 seconds
```

```
-----
Seconds      %      Task name
-----
```

```
106.6842293  100%
```

[Объяснить с](#) 

Выставляем пачку больше, чем имеем сейчас, давайте сделаем 5000 и получаем результат $\approx 1.86\%$ быстрее, неплохо, хотя мы просто поменяли 1 параметр:

```
StopWatch '': 96.580727916 seconds
```

```
-----  
Seconds      %      Task name  
-----
```

```
96.58072792   100%
```

[Объяснить с 🧑🏻💻](#)

Я не буду искать прям самый лучший вариант, тут хотел показать возможность еще такой оптимизации.

Шаг 4. Горизонтальное масштабирование системы

В начале статьи я сказал, что у нас всего 1 инстанс, все понимают, это неправильно, и нужно масштабироваться. Но все не так просто, иногда код просто к этому не готов.

Давайте напишем простую проверку, добавим AtomicInteger для подсчета количества походов в смежную систему

```
public static final AtomicInteger counter = new AtomicInteger();  
  
public RegistrationResponseDto registrationUser(RegistrationDto dto) {  
    counter.incrementAndGet();  
    LockSupport.parkNanos(TimeUnit.MILLISECONDS.toNanos(1));  
    return new RegistrationResponseDto(UserStatus.SUCCESS.name(), null);  
}
```

[Объяснить с 🧑🏻💻](#)

Запустим все это в 2 потоках(имитация двух инстансов), отдельным классом, который имплементирует ApplicationRunner:

```
@Override  
public void run(ApplicationArguments args) throws Exception {  
    var t1 = new Thread(processService::sendUserToRegistration);  
    var t2 = new Thread(processService::sendUserToRegistration);  
    t1.start();  
    t2.start();  
    t1.join();  
    t2.join();  
    System.out.println("Кол-во походов " + OtherSystemClient.counter);  
}
```

[Объяснить с 🧑🏻💻](#)

```
StopWatch '': 96.307432834 seconds
```

```
-----  
Seconds      %      Task name  
-----
```

```
96.30743283   100%
```

```
StopWatch '': 96.344891667 seconds
```

Seconds	%	Task name
96.34489167	100%	

Кол-во походов 150000

Объяснить с 

Мало того, что мы все выполнили 2 раза, так еще и непонятно, как это повлияет на нашу систему.

Нужно это исправить, есть несколько подходов, я же буду использовать - **SELECT FOR UPDATE SKIP LOCKED**

У этого подхода есть свой минус, транзакция держится все время, даже на время REST вызова, что очень плохо, но при таком подходе откат будет предсказуемым и правильным.

Вы можете использовать подход со статусом PROCESSING и изначальной установкой этого статуса:

```
@Service
@RequiredArgsConstructor
public class UserBatchService {

    private final UserRepository userRepository;

    @Transactional
    public List<User> markProcessing(Long lastId) {
        return userRepository.markBatchAsProcessing(UserStatus.NEW.name(), lastId);
    }

    @Transactional
    public void saveBatch(Collection<Long> ids, String status) {
        userRepository.batchUpdateStatus(ids, status);
    }
}
```

Объяснить с 

FOR UPDATE подход:

Я контролирую процесс целиком

PROCESSING подход:

Я контролирую состояние, а не процесс

Каждый сам выбирает то, что ему нужно.

Продолжаем для нашего **SELECT FOR UPDATE SKIP LOCK**

Заменяем наш метод **findAllByStatus** на такой:


```

@Query(value = """
    SELECT * FROM users
    WHERE status = :status AND id > :lastId
    ORDER BY id
    LIMIT 5000
    FOR UPDATE SKIP LOCKED
    """, nativeQuery = true)
List<User> findAndLockByStatus(@Param("status") String status, @Param("lastId") Long

```

[Объяснить с 📖](#)

И теперь меняем основной метод на:

```

@Transactional
public void sendUserToRegistration() {
    Stopwatch stopWatch = new Stopwatch();
    stopWatch.start();

    Long lastId = 0L;           // Курсор для постраничной загрузки
    List<User> batch;           // Текущая пачка пользователей
    int totalProcessed = 0;     // Счетчик обработанных записей

    do {
        // 1. Атомарно захватываем пачку из 5000 записей
        // FOR UPDATE SKIP LOCKED гарантирует, что:
        // - записи блокируются от изменений другими транзакциями
        // - другие потоки пропускают уже заблокированные записи
        batch = userRepository.findAndLockByStatus(UserStatus.NEW.name(), lastId);

        // 2. Если новых записей нет — выходим из цикла
        if (batch.isEmpty()) {
            break;
        }

        // 3. Группируем пользователей по целевому статусу
        Map<UserStatus, List<Long>> usersByStatus = batch.stream()
            .collect(Collectors.groupingBy(
                user -> {
                    RegistrationDto dto = userMapper.toRegistrationDto(user);
                    RegistrationResponseDto response = otherSystemClient.registrationUs
                    return UserStatus.valueOf(response.getStatus());
                },
                Collectors.mapping(User::getId, Collectors.toList())
            ));

        // 4. Одним запросом обновляем статусы для каждой группы
        // Используем batch UPDATE через ANY(:ids) для производительности
        usersByStatus.forEach((status, ids) ->
            userRepository.batchUpdateStatus(ids, status.name())
        );

        // 5. Обновляем счетчики и курсор
        totalProcessed += batch.size();
        lastId = batch.getLast().getId(); // Запоминаем последний ID для следующей итерации
    }
}

```

```

    } while (batch.size() == 5000); // Продолжаем, пока пачка полная (есть еще данные)

    System.out.println("Всего обработали " + totalProcessed);
    stopWatch.stop();
    System.out.println(stopWatch.prettyPrint());
}

```

[Объяснить с 🧑🏻💻](#)

Новый метод выглядит так, давайте его запустим в 2 потоках и посмотрим на результат:

```

StopWatch '': 44.914883375 seconds
-----
Seconds      %      Task name
-----
44.91488338   100%
StopWatch '': 51.318702625 seconds
-----
Seconds      %      Task name
-----
51.31870263   100%

Кол-во походов 75000

```

[Объяснить с 🧑🏻💻](#)

Отлично, видим хороший результат 51 секунда $\approx$ 46.7% быстрее, чем было!

Давайте на всякий случай проверим, если ли дубли у нас:

```

public static final AtomicInteger counter = new AtomicInteger();
public static final ConcurrentHashMap<String, AtomicInteger> emailCounter = new Concu

public RegistrationResponseDto registrationUser(RegistrationDto dto) {
    counter.incrementAndGet();
    LockSupport.parkNanos(TimeUnit.MILLISECONDS.toNanos(1));
    emailCounter.computeIfAbsent(dto.getEmail(), k -> new AtomicInteger())
        .incrementAndGet();

    return new RegistrationResponseDto(UserStatus.SUCCESS.name(), null);
}

```

[Объяснить с 🧑🏻💻](#)

И выведем все это:

```

System.out.println(OtherSystemClient.emailCounter.size());
System.out.println(OtherSystemClient.emailCounter.entrySet().stream()
    .filter(it -> it.getValue().get() > 1)
    .map(Entry::getKey)
    .toList());

```

[Объяснить с 🧑🏻💻](#)

Получаем результат:

```
75000  
[]
```

[Объяснить с 🧑🏻💻](#)

Отлично, мы не отправили ничего лишнего, хорошие новости.

Я прекрасно понимаю, что 2 потока в одном java приложении != 2 инстанс, так давайте это исправим. Нам всего лишь нужно запустить 2 наших приложения и вывести логи.

Добавляем в application.yml:

```
logging:  
  file:  
    name: logs/app-${spring.application.name}.log
```

[Объяснить с 🧑🏻💻](#)

Запускаем 2 наших приложения

```
# Инстанс 1  
java -jar путь_до_jar --server.port=8080 --spring.application.name=app-1 > app1.log 2>&  
  
# Инстанс 2  
java -jar путь_до_jar --server.port=8081 --spring.application.name=app-2 > app2.log 2>&
```

[Объяснить с 🧑🏻💻](#)

Получаем в 2 файлах логов 2 записи:

```
Всего обработано: 35806  
StopWatch '': 45.961987833 seconds  
-----  
Seconds      %      Task name  
-----  
45.96198783  100%
```

[Объяснить с 🧑🏻💻](#)

```
StopWatch '': 49.562074708 seconds  
-----  
Seconds      %      Task name  
-----  
49.56207471  100%
```

[Объяснить с 🧑🏻💻](#)

Результат сложения: **75 000**.

Это ровно то количество записей (75 000), которое у нас было изначально со статусом `NEW`. Мы получили ≈ 50 секунд в логах, супер результат, по сравнению с начальными показателями мы ускорились $\approx$ на 52%

Шаг 5. Добавляем потоки

Мы понимаем, что в нашей системе поход в смежную систему можно выделить в отдельные потоки, так давайте это сделаем.

Начинаем с добавления `Executor`:

```
@Bean
public Executor registrationExecutor() {
    return Executors.newFixedThreadPool(10); // выбирайте сами, рекомендую из конфига
}
```

Объяснить с 

Затем его внедряем и добавляем метод обработки:

```
public record ProcessedUser(Long id, UserStatus status) {
}
```

Объяснить с 

```
private final Executor registrationExecutor;
...бла бла бла

private Map<UserStatus, List<Long>> processBatchInParallel(List<User> batch) {

    List<CompletableFuture<ProcessedUser>> futures = batch.stream()
        .map(user -> CompletableFuture.supplyAsync(() -> {

            RegistrationDto dto = userMapper.toRegistrationDto(user);
            RegistrationResponseDto response =
                otherSystemClient.registrationUser(dto);

            return new ProcessedUser(
                user.getId(),
                UserStatus.valueOf(response.getStatus())
            );

        }, registrationExecutor))
        .toList();

    List<ProcessedUser> results = futures.stream()
        .map(CompletableFuture::join)
        .toList();

    return results.stream()
        .collect(Collectors.groupingBy(
            ProcessedUser::status,
```

```

        Collectors.mapping(ProcessedUser::id, Collectors.toList())
    ));
}

```

[Объяснить с 🧑🏻💻](#)

Наш целый метод:

```

@Transactional
public void sendUserToRegistration() {
    Stopwatch stopWatch = new Stopwatch();
    stopWatch.start();

    Long lastId = 0L;           // Курсор для постраничной загрузки
    List<User> batch;           // Текущая пачка пользователей
    int totalProcessed = 0;     // Счетчик обработанных записей

    do {
        batch = userRepository.findAndLockByStatus(UserStatus.NEW.name(), lastId);

        if (batch.isEmpty()) {
            break;
        }

        Map<UserStatus, List<Long>> usersByStatus = processBatchInParallel(batch);

        usersByStatus.forEach((status, ids) ->
            userRepository.batchUpdateStatus(ids, status.name())
        );

        // 5. Обновляем счетчики и курсор
        totalProcessed += batch.size();
        lastId = batch.getLast().getId();
    } while (batch.size() == 5000);

    System.out.println("Всего обработали " + totalProcessed);
    stopWatch.stop();
    System.out.println(stopWatch.prettyPrint());
}

```

[Объяснить с 🧑🏻💻](#)

Получаем результат в 2 экземплярах:

```

Всего обработали 40000
StopWatch '': 5.74660275 seconds
-----
Seconds      %      Task name
-----
5.74660275   100%

```

[Объяснить с 🧑🏻💻](#)

```
Всего обработали 35000
StopWatch '': 4.98958625 seconds
```

Seconds	%	Task name
4.98958625	100%	

[Объяснить с 🧑🏻💻](#)

Относительно прошлого шага мы выросли $\approx 88.4\%$, что очень круто!

А теперь я предлагаю "Ультануть", если мы используем 21+ java, то мы можем использовать **виртуальные потоки**. Давайте чуток поменяем наш бин:

```
@Bean
public Executor registrationExecutor() {
    return Executors.newVirtualThreadPerTaskExecutor(); // виртуальные потоки
}
```

[Объяснить с 🧑🏻💻](#)

Запускаем:

```
Всего обработали 35000
StopWatch '': 0.84364875 seconds
```

Seconds	%	Task name
0.84364875	100%	

[Объяснить с 🧑🏻💻](#)

```
Всего обработали 40000
StopWatch '': 0.879780375 seconds
```

Seconds	%	Task name
0.879780375	100%	

[Объяснить с 🧑🏻💻](#)

Получаем результат < 1 секунды, ну что за красота. Это рост на 98.4% относительно 2 инстансов! Для тех, кто не знаком с виртуальными потоками, очень советую с ними познакомиться.

[Моя статья по виртуальным потокам](#)

Тут очень важно, чтобы смежная система выдержала нагрузку, которую мы хотим ей дать. Может быть такое, что у нас все летает, но RPS внешней системы 5-10 запросов.

Итоги

Сегодня мы увидели, как пошаговая работа с узкими местами меняет картину целиком: **с 1 минуты 45 секунд до менее чем одной секунды**. Позади - анализ планов выполнения, борьба с блокировками и настройка батчей.

Конечно в этом коде нет обработки ошибок, ретраев и тд, так как в этой статье я хотел показать подход с оптимизацией.

Помните: легаси - это просто код, который еще не успели полюбить и оптимизировать. Подключайте мониторинг, не бойтесь переписывать горячие участки и доверяйте цифрам, а не ощущениям.

Рад был поделиться опытом. Всем легкого легаси и хорошего дня!

Теги: [java](#), [kotlin](#), [spring](#), [spring boot](#), [оптимизация](#), [оптимизация кода](#), [многопоточность](#), [postgresql](#), [hibernate](#), [legacy](#)

Хабы: [Java](#), [Kotlin](#)



Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



4K+

16

Охват за 30 дней

Карма

@MishaBucha

Пользователь

Подписаться



Поток Бэкенд доступен 24/7 благодаря поддержке друзей Хабра

Хабр Курсы для бэкендеров

РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

[Перейти в поток Бэкенд](#)

Комментарии 2

Публикации

[ЛУЧШИЕ ЗА СУТКИ](#)

[ПОХОЖИЕ](#)



the_bat

23 часа назад

Ремонт блока питания с Power Delivery. 470 граммов электроники

 Простой  4 мин  16K

 +97

 31

 27

 **interpres**

18 часов назад

Электроинструмент становится хуже, и это делается намеренно

 Простой  11 мин  27K

Обзор

Перевод

 +72

 45

 87

 **prohronus**

19 часов назад

Как ИИ-агенты стали новым оружием скамеров на Хабр Карьере

 5 мин  9.4K

Кейс

 +52

 9

 9

 **alizar**

22 часа назад

Готовимся к отключению. Эффективные форматы для упаковки и раздачи HTML-страниц

 Простой  7 мин  12K

Обзор

 +36

 60

 18

 **AlexSerbul**

23 часа назад

Программирование с AI-ассистентом — похороните меня под плинтусом

 Средний  14 мин  10K

Мнение

 +28

 33

 11

 **TrexSelectel**

19 часов назад

Linux 7.0 и что изменилось в ядре после очередного цикла разработки

 5 мин  8.9K

Обзор

 +27

 10

 0

 **mariklolik**

19 часов назад

Как переложить нагрузку по code review с разработчиков на LLM

Средний 7 мин 7.6K

Кейс

+23

23

6

nick_dyuba
23 часа назад

Легенды 90-х — кто придумал и производил жвачки Turbo, Love is... и TіріTіr

5 мин 7.1K

Recovery Mode

+22

8

5

infgotoinf
11 часов назад

Perl — зря забытый язык программирования?

11 мин 8.7K

Из песочницы

+16

13

10

SLY_G
22 часа назад

Что такое «мышечная память» и можно ли её развить?

4 мин 7.1K

Перевод

+16

10

7

Тысяча нюансов: как управлять производством лекарств

Турбо

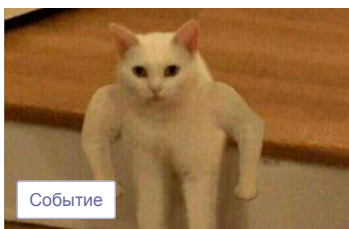
Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Турбо

Собери облачный пазл и выиграй призы



Событие

Качаем к лету хард- и софт-скиллы на айтишных ивентах



Турбо

Удалёнка в фарме: большие данные и никакой магии

ВАКАНСИИ

Android разработчик / Kotlin developer

от 120 000 до 180 000 Р · Hіgeway · Можно удаленно

Java Developer

от 75 000 Р · ИТK academy · Казань · Можно удаленно

Java разработчик с нуля (стажер)

от 52 000 Р · Aston · Москва · Можно удаленно

Java Developer

от 75 000 Р · ИТРУМ · Ростов-на-Дону · Можно удаленно

SEO-специалист

от 1 000 до 1 500 \$ · MST · Можно удаленно

[Больше вакансий на Хабр Карьере](#)

ЧИТАЮТ СЕЙЧАС

Электроинструмент становится хуже, и это делается намеренно

 27K  87 

Тим Кук покидает пост Apple. Новый глава — «отец» Apple Silicon Джон Тернус

 14K  16 

Представлен проект KillerPDF — редактор PDF с открытым исходным кодом для Windows 10/11

 7.4K  16 

Пользователь купил Atomic Heart на VK Play, но не смог запустить игру из-за сетевых проблем

 5.8K  12 

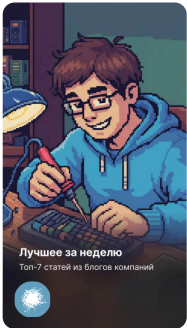
+185% за 13 часов: как Kimi K2.6 переписала 8-летний движок

 15K  11 

Тысяча нюансов: как управлять производством лекарств

Турбо

ИСТОРИИ



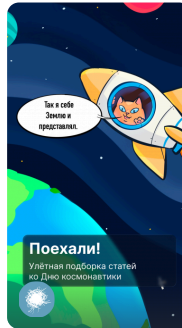
Годнота из блогов компаний



Только 10% решат эту головоломку



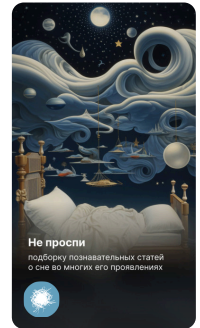
Хочу 450K



Вперёд к звёздам



Там на неведомых дорожках...



Самое время поспать

БЛИЖАЙШИЕ СОБЫТИЯ



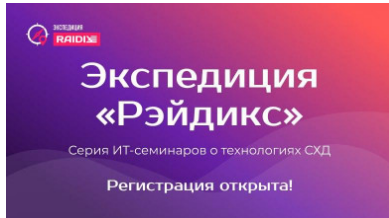
2 марта – 10 августа

**PROBA Awards —
международная
коммуникационная
премия, учрежденная в
2000 году**

Санкт-Петербург

Маркетинг

[Больше событий в календаре](#)



3 марта – 9 июня

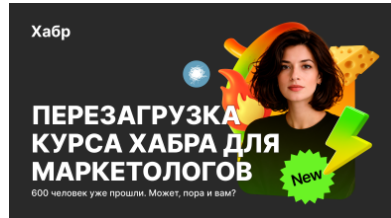
Экспедиция «Рэйдикс»

Екатеринбург • Новосибирск •
Краснодар • Минск • Калининград •
Ростов-на-Дону

Разработка

Другое

[Больше событий в календаре](#)



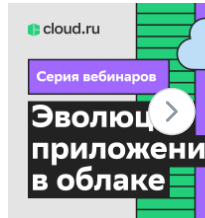
9 марта – 1 июня

**Всё, что нужно, чтобы
стартовать продвижение
бренда на Хабре**

Онлайн

Маркетинг

[Больше событий в календаре](#)



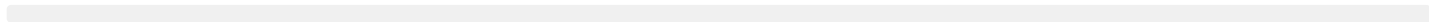
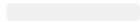
19 марта – 28 апреля

**Серия вебинаров
«Эволюция п
в облаке»**

Онлайн

Разработка

[Больше событий в к](#)



Ваш аккаунт

Войти
Регистрация

Разделы

Статьи
Новости
Хабы
Компании
Авторы
Песочница

Информация

Устройство сайта
Для авторов
Для компаний
Документы
Соглашение
Конфиденциальность

Услуги

Корпоративный блог
Медийная реклама
Нативные проекты
Образовательные программы
Стартапам



Настройка языка

Техническая поддержка

© 2006–2026, Habr